

DSCI 511 Quiz 2 Cheat Sheet

Pandas: inspect, select, transform

```
import pandas as pd
df = pd.read_csv("data.csv") # sep="\t",
header=None, names=[...], index_col=...
df.shape; df.columns.to_list(); df.head(); df.info();
df.describe()
s = df["col"] # Series;
df[["col"]] = DataFrame
x = df.loc[row_label, col_label] #
labels; .iloc[row_pos, col_pos] = 0-based positions
row = df.loc[key] # list keeps
2-D
```

- .shape/.columns are attributes. set_index returns new frame: assign.
- Vectorized columns align by index: df["scaled"] = df.score * 1.03; avoid element loops.
- axis=0 (default) reduces to per-column; axis=1 to per-row (mean, idxmax).
- Filter: m = (df.region == "West") & (df.year > 2015); use parenthesized comparisons with & | ~ (not and/or/not). query("year > 2015 and region == 'West'") permits and/or/not; backtick spaced names. Masks also support .isin()/.str.
- sort_values(by=..., ascending=False); s.value_counts() frequency; s.nunique() distinct count.

Missingness, edits, reshape

```
missing = df.isna().any() # NaN != ""
or 0; absent rows may hide missingness
clean = df.dropna() # or
fillna(value); returns new object
clean = df.rename(columns={"old":
"new"}).drop(columns=["unused"])
df.to_csv("clean.csv", index=False) # choose
whether to write index
```

- Most edits/reshapes return new objects: assign drop/fillna/rename/sort_values/set_index/reset_index. Copy first if preserving input: out = df.copy().
- df.columns=[...] replaces all names (exact length/order); use rename for a few.
- Tidy: one variable/column, observation/row, observational unit/table. Define row meaning before reshape.

Task	Operation / trap
Long→wide, unique keys	pivot(index=..., columns=..., values=...); duplicate pairs error
Long→wide, duplicate pairs	pivot_table(..., aggfunc="mean"); aggregates, doesn't keep one
Wide→long	melt(id_vars=[...], value_vars=[...], var_name=..., value_name=...); omitted value_vars melts all other columns
Axes/index	transpose() swaps axes; reset_index() makes index a column (new frame)

Pivot NaN may mean absent combination; fill 0 only if semantically zero.

Combine, apply, group

```
stack = pd.concat([a, b]) # axis=0
stacks; axis=1 aligns index
both = pd.merge(a, b, how="left", on="id") # or
left_on=..., right_on=...
by_index = pd.merge(a, b, left_index=True,
right_index=True)
col_sums = df[["a", "b"]].apply(sum) # axis=0
columns; axis=1 rows (each Series)
cells = df.map(func) # each
scalar; s.apply(func) uses Series
mapped = s.map(mapping) # dict/
function; unmapped keys → NaN
summary = df.groupby(["country",
"year"]).agg(total=("score", "sum"))
```

- concat joins along axis, not business key; unequal columns create NaN, duplicate labels may remain.
- merge: shared key on, differing names left_on/right_on; left/right/outer control retained keys. Column merge resets index.
- Check key meaning/cardinality: duplicate keys can cause many-to-many row multiplication. groupby splits then aggregates (sum/mean/count/agg); multi-key gives combined index; select relevant columns, reset_index() for columns.
- Prefer vectorization. apply for whole rows/columns; map for scalar mapping.

Strings, datetime, categories

```
parts = df.Genre.str.split(",", expand=True); parts[1] =
parts[1].str.strip()
df["Date"] = pd.to_datetime(df.Date)
df = df.set_index("Date").sort_index()
period = df.loc["2019-10"] # range
slices: sort index first
daily = df[["Time", "Distance"]].resample("1D").mean() #
aggregate required
hour = df.between_time("00:00", "01:00")
years = df.Date.dt.year # Series
uses .dt, not .year
level = pd.Categorical(df.level, ["low", "medium",
"high"], ordered=True)
```

- Series strings use .str; datetime Series .dt. DatetimeIndex: .weekday/.day_name(); Timestamp point, Period interval, Timedelta duration, missing = NaT. date_range/period_range build ranges.
- Resample frequencies: 1D day, h hour, min minute, B business day, MS/ME month start/end; empty bins may be NaN.
- category suits finite stable values, can save memory and encode order. Clean spelling/case first; declared category order controls sorting.

Streaming, tokens, datasets

```
def lines(path):
    with open(path) as f:
        for line in f:
            line = line.strip()
            if line: yield line
```

- Generator body runs on next()/for, pauses at yield; one-use, forward-only (no index/len). list(gen) consumes into RAM; exhausted next() raises StopIteration. Stream large data. with open closes file even if iteration stops early.
- Tokenization: split text; dict maps each new token to integer ID (vocab[token] = len(vocab)); yield ID sequences. Dataset defines data/access (iterable sequential; map-style indexed); DataLoader samples it into batches, with batch size/optional shuffle.

```
import numpy as np
```

```
def batches(sequences, batch_size):
    pending = []
    for ids in sequences:
        pending.append(ids)
        if len(pending) == batch_size:
            width = max(map(len, pending))
            batch = np.zeros((len(pending), width)) # pad
            for i, ids in enumerate(pending):
                batch[i, :len(ids)] = ids
            yield batch
            pending = []
    # Full-batch-only template drops final short batch;
    handle remainder if required.
```

numpy.memmap: disk random access

```
mm = np.memmap("tokens.memmap", dtype=np.int32, mode="w+",
shape=(n,))
mm[start:stop] = token_ids; mm.flush() # persist
writes
chunk = mm[i * seq_len:(i + 1) * seq_len]
```

Disk-backed array supports indexed/slice access without loading whole file; useful for map-style random sampling/shuffle. Match dtype/shape to layout. Check 0 <= i < len(dataset): memmap bounds are limited; enforce in dataset __len__/_getitem__.

Tests and debugging

```
import math
assert result == expected
assert math.isclose(actual, expected, abs_tol=0.001) #
float tolerance, not ==
```

- AAA: Arrange → Act → Assert. Keep tests simple; compare members if order irrelevant. Cover corners (empty, one item, even/odd, boundaries); passing suite doesn't prove completeness.
- TDD: stub → specify/test → pseudocode → small implementation/test steps → document (course practice, not mandatory).
- EAFF: try, catch expected error (d["address"], except KeyError); LBYL: check first (if "address" in d). Choose by context.